

Projet Innovant · WSF5

Promotion 2026

DREAMWEAVE

NarraMind

Système de mémoire narrative & cohérence scénaristique :
maintenir un webtoon cohérent sur des dizaines de chapitres
sans saturer le contexte d'un LLM

Louis Basnier

Posture CTO · Étude, conception & prototypage POC (v3)

Introduction

Qu'est-ce que DreamWeave ?

DreamWeave est un outil web de création de webtoons (bandes dessinées numériques à défilement vertical, très populaires en Corée puis mondialement) assisté par intelligence artificielle. Sa promesse : permettre à quelqu'un qui a une histoire à raconter mais aucune compétence en dessin de produire une série illustrée cohérente et publiable.

Le marché du webtoon croît d'environ 27 %/an, mais le nombre de créateurs capables de dessiner ne suit pas. DreamWeave s'attaque à cette barrière : générer des visuels cohérents en quelques secondes, sans savoir illustrer.

Le parcours de création en 5 étapes

L'utilisateur avance dans un pipeline linéaire. Chaque étape produit la matière de la suivante.

- 01 Style**
Définir l'ADN visuel de la série à partir d'une image de référence, pour que tous les visuels partagent la même patte graphique.
- 02 Assets**
Générer les personnages, décors et objets, visuellement cohérents entre eux grâce au Sheet System (fiche composite 4 angles).
- 03 Univers**
Construire le lore du monde (règles, lieux, factions, personnages, chronologie) : la « bible » de la série qui sert de référence de cohérence. C'est ici qu'Ariane enrichit l'Univers à partir du scénario.
- 04 Scénario**
Écrire l'histoire chapitre par chapitre (assisté par l'IA scénariste), puis la découper en cases.

Composer chaque « case » (planche) : blocs d'image, blocs de couleur et bulles de dialogue.

Étapes ① & ②

Le visuel : déjà résolu

La génération d'images cohérentes (un même personnage reconnaissable de case en case) repose sur FLUX.2 Pro (FAL.ai) et le Sheet System (fiche 4 angles). C'est la valeur d'origine du produit, et elle est livrée.

Étapes ③ & ④

Le monde & le récit : le vrai point dur

Générer une belle image est déjà un problème résolu. Garder une histoire cohérente sur des dizaines de chapitres ne l'est pas : éviter qu'un personnage mort réapparaisse, ne pas contredire une règle du monde posée 30 chapitres plus tôt... C'est ce problème-là qu'on attaque ici.

Le problème résolu par NarraMind

Sur le pipeline ci-dessus, DreamWeave dispose déjà d'une IA scénariste (le module generate-scenario-ai) qui aide à écrire les chapitres. Mais un modèle de langage (LLM) a une mémoire limitée : il ne peut pas relire tout un roman à chaque phrase générée. Plus l'histoire grandit, plus il « oublie » le début, ou coûte trop cher et devient trop lent si on lui réinjecte tout.

NARRAMIND

NarraMind est une couche invisible de mémoire narrative greffée sur l'IA scénariste.

Son rôle : donner au LLM un souvenir compact mais fidèle de toute l'histoire, détecter les incohérences, et proposer des pistes cohérentes, sans jamais ralentir ni interrompre l'auteur. C'est la brique la plus complexe de DreamWeave.

NarraMind pense, Ariane parle. L'utilisateur ne voit jamais « NarraMind » dans l'application : c'est le moteur invisible qui analyse le texte et se souvient. Ce qu'il détecte est retranscrit à l'auteur par **Ariane**, un personnage-guide affiché à l'écran qui formule les incohérences et

les suggestions en langage « histoire ». Ariane est donc l'interface visible de NarraMind, les deux noms désignent les deux faces (moteur / interface) d'un même système.

Glossaire : les termes du dossier

Tableau de référence : chaque terme technique ou nom propre utilisé plus loin est défini ici, en langage simple.

Terme	Ce que ça veut dire, simplement
NarraMind	Le moteur invisible de mémoire narrative. Il analyse le scénario, se souvient de l'histoire de façon compacte et repère les incohérences. L'utilisateur ne le voit jamais directement.
Ariane	L'entité visible à l'écran (personnage-guide) qui retranscrit à l'auteur ce que NarraMind a identifié : incohérences et suggestions, en langage « histoire », jamais en jargon technique. Résumé : NarraMind pense, Ariane parle.
Compass (NarraMind Compass)	L'extension proactive de NarraMind : au lieu de seulement détecter les erreurs, elle propose des idées cohérentes (directions de scénario, enrichissement du lore) en s'appuyant sur la vectorisation.
Lore	L'ensemble des règles et éléments du monde inventé par l'auteur : personnages, lieux, factions, magie, chronologie. La « bible » de la série.
LLM & « tokens »	Un LLM (comme Gemini) lit et écrit du texte découpé en tokens ($\sim \frac{3}{4}$ de mot). Sa mémoire de travail est bornée : plus on lui donne de tokens, plus c'est lent et cher. Réduire les tokens en gardant le sens est l'enjeu n°1.
Vectorisation, embedding & RAG	Vectoriser un texte = le transformer en une liste de nombres (un embedding) qui capture son sens. Deux textes de sens proche ont des embeddings proches. Le RAG ne donne au LLM que les quelques passages les plus pertinents, retrouvés ainsi.

Terme	Ce que ça veut dire, simplement
pgvector	L'outil qui stocke ces embeddings dans la base de données et retrouve les plus proches en quelques millisecondes, sans logiciel externe à ajouter.
Sheet System	La fiche d'un personnage ou décor sous 4 angles, qui garantit qu'il reste reconnaissable d'une image à l'autre.
POC	Proof of Concept : un prototype qui sert à prouver qu'une idée difficile est réalisable, sans construire le produit complet.
LM OPS	L'ensemble des pratiques pour surveiller et fiabiliser une IA en production : mesurer ce qui entre et sort du modèle, poser des garde-fous.
MAU	Monthly Active Users : nombre d'utilisateurs actifs par mois. Sert à estimer les coûts à l'échelle.
RLS	Règle de sécurité de la base garantissant que chaque utilisateur ne voit que ses propres données.

Sommaire

01	Projet	p.7
02	Description fonctionnelle	p.9
03	Modélisation technique & benchmarks	p.10
04	Méthodologie de prototypage du POC	p.14
05	Documentation technique & budget	p.23
06	Plan de release	p.26

01 Projet

1.1 Présentation du projet

NarraMind est le moteur de mémoire narrative de DreamWeave. Son rôle : permettre à une IA scénariste de maintenir la cohérence d'un webtoon sur des dizaines de chapitres, sans jamais saturer la fenêtre de contexte d'un LLM.

Il s'insère dans DreamWeave comme une couche invisible : l'utilisateur n'interagit pas avec NarraMind directement, il écrit son scénario, et si une incohérence est détectée, le personnage-guide **Ariane** la lui signale dans un panneau dédié, en langage « histoire » (jamais en jargon IA).

NarraMind n'est pas une feature de génération. C'est une couche de **mémoire** et de **cohérence**, précisément la brique technique la plus difficile du produit, donc le sujet légitime d'un Projet Innovant.

1.2 Objectifs du projet

#	Objectif	Cible mesurable
O1	Maintenir un contexte LLM stable quel que soit le nombre de chapitres	$\leq 1\,400$ tokens de contexte compressé au chapitre 20
O2	Détecter les incohérences narratives (personnage décédé qui réapparaît, lore contradictoire...)	Anomalies détectées et persistées dans <code>narramind_alerts</code>
O3	Présenter les alertes à l'auteur sans l'interrompre	Déclenchement invisible, panneau Continuité consulté sur demande
O4	Conserver le lore de l'univers sur le long terme	<code>memory_entities</code> , <code>memory_summaries</code> , <code>narra_summary</code> cohérents à 50 chapitres

1.3 Enjeux

Enjeu technique

La fenêtre de contexte LLM

Un LLM a une limite de tokens. Injecter brut l'intégralité des chapitres précédents croît linéairement (~850 tokens/chapitre). Dès 10 chapitres, la latence double et les coûts explosent. À 50 chapitres, l'approche naïve est inutilisable. NarraMind résout ça par compression multi-niveaux.

Enjeu UX

Ne jamais bloquer l'auteur

NarraMind ne doit ni ralentir ni interrompre l'écriture. Déclenchement automatique, pas de bouton « Vérifier », pas d'écran de données brutes. L'auteur n'a conscience du système que quand Ariane a quelque chose d'utile à dire.

Enjeu de cohérence

Lore vs. histoire

Les alertes sont rédigées en langage auteur (personnages, événements, lieux), jamais en vocabulaire technique (asset_id, memory_entity). La confiance de l'auteur dépend de la pertinence des alertes.

Enjeu de propriété

L'œuvre reste à l'auteur

NarraMind ne revendique aucune propriété. Le LORE est écrit par l'utilisateur ; le système ne fait que des suggestions traçables (extrait de l'histoire vs. inventé), que l'auteur valide ou rejette.



Description fonctionnelle

À quoi ressemble NarraMind une fois terminé, et quelle expérience utilisateur produit-il ?

2.1 Expérience utilisateur attendue

Du point de vue de l’auteur, NarraMind est invisible par défaut. Le parcours nominal ne comporte aucune étape technique visible.

- L’auteur ouvre l’éditeur de chapitre et écrit son texte.
- À l’auto-save (≥ 80 mots), NarraMind s’exécute silencieusement en arrière-plan.
- Si une incohérence est détectée, le bouton **Continuité** (icône Ariane) l’indique dans l’éditeur.
- L’auteur ouvre le panneau Continuité : liste d’alertes (titre + explication en langage histoire + sévérité).
- Il marque chaque alerte « Traitée » ou « Ignorer », la décision finale lui appartient toujours.
- Aucun écran de « fiches entités » ni de « résumés techniques » n’est exposé.

NarraMind s’exécute aussi à la sauvegarde du lore monde (onglet Univers), pour propager les nouvelles règles au dernier chapitre analysé.

2.2 Résultats attendus

Résultat	Indicateur
Contexte LLM compressé et stable	≤ 1 400 tokens au chapitre 10, stable ensuite
Croissance maîtrisée	~50 tokens/chapitre (vs. ~850 en baseline)
Alertes exploitables	Langage histoire, titre + explication, sévérité low/medium/high
Persistance des alertes	Table narramind_alerts avec statuts active/dismissed/resolved
Mémoire longue fiable	narra_summary mis à jour par arc + compression par lots des résumés anciens

2.3 Ce que NarraMind ne fait pas

- Il ne bloque jamais l'écriture.
- Il ne génère pas le scénario (c'est le rôle de generate-scenario-ai).
- Il n'expose pas les données brutes de mémoire à l'utilisateur.
- Il ne revendique pas la propriété de l'œuvre, le LORE est écrit par l'utilisateur.

03 Modélisation technique

3.1 Approche algorithmique : compression multi-niveaux

Le problème central : comment fournir à un LLM un contexte narratif riche mais borné, quel que soit le nombre de chapitres écrits ? La réponse est une architecture mémoire à 3 niveaux.

NIVEAU 1 : Fiches entités (memory_entities)

- Upsert par nom : les entités sont mises à jour, jamais dupliquées
- Croissance $O(1)$ en tokens (50 personnages = ~300 tokens fixes)

NIVEAU 2 : Résumés glissants (memory_summaries)

- ~80 tokens par chapitre
- Compression par lots : les 8 résumés les plus anciens sont fusionnés par le LLM quand le seuil de tokens est atteint

NIVEAU 3 : Méga-résumé projet (narra_summary)

- Consolidation périodique des arcs passés
- Stocké en BDD, tronqué au prompt si dépassement

Budget du prompt construit à chaque run

[Méga-résumé projet]	~400 tokens
[Résumés récents chapitres]	~300 tokens (fenêtre bornée)
[Entités mémoire]	~300 tokens (priorité : récentes + mentionnées)
[Universe lore]	tronqué si > 14k caractères
[Chapitre courant intégral]	toujours entier

Total cible ≤ 1 400 tokens (hors chapitre courant)

Sortie LLM en JSON structuré : { entities_to_update, chapter_summary, anomalies[] }. Les anomalies sont immédiatement enregistrées dans narramind_alerts en évitant les doublons (clé dedupe_key) ; les alertes qui n'apparaissent plus dans une nouvelle analyse passent automatiquement en « résolues ». Ainsi, ré-analyser le même chapitre ne crée jamais de doublon.

3.2 Déclenchement automatique avec garde-fous

ScenarioChapterEditor

- auto-save réussi + ≥ 80 mots + délai mini 12 min (même chapitre)
- triggerNarraMindUpdate(projectId, chapterId)

UniverseSection

- sauvegarde lore monde ou LORE asset
- triggerNarraMindUpdate(projectId, dernierChapitre)

Pas de bouton « Vérifier », uniquement du déclenchement automatique borné.

3.3 Benchmark technologique & justifications

Cinq critères ont été définis puis pondérés selon leur impact sur la contrainte centrale (contexte borné, latence faible, coût maîtrisé à l'échelle).

Critère	Pondération	Définition
Coût par token	30 %	Prix input + output pour 1M tokens (API publique)
Vitesse de génération	25 %	Tokens/seconde en conditions réelles (réponse JSON ~600 tokens)
Qualité JSON structuré	25 %	Respect d'un schéma strict sans hallucination de champ
Quotas API	10 %	Accès gratuit ou généreux pour le développement et les tests
Compatibilité écosystème	10 %	Compat. OpenAI-API, intégration native dans la stack

Benchmark LLM principal

	Gemini 2.x Flash 	GPT-4o	Groq Llama 3.3 70B
Coût input (1M tok.)	\$0,10	\$2,50	\$0,59
Coût output (1M tok.)	\$0,40	\$10,00	\$0,79
Vitesse (tok./s)	~100	~50	~200
Qualité JSON	★★★★	★★★★★	★★★
Quotas gratuits	1M tok./jour	Aucun	Généreux
Score pondéré	 4,4 / 5	3,1 / 5	3,8 / 5

Décision : LLM

Retenu : Gemini 2.0 → 2.5 Flash en principal, Groq Llama 3.3 70B en fallback.
Justification : GPT-4o écarté (25× plus cher pour une qualité JSON pas décisive à ce niveau de détail). Groq gardé en secours pour sa vitesse.

Benchmark runtime Edge Function

	Deno / Supabase 	Node.js Lambda	Cloudflare Workers
Cold start	< 50 ms	100-500 ms	< 5 ms
Intégration Supabase	Native	Via SDK	Via SDK
Coût (500K inv./mois)	Inclus	~\$0,10	~\$0,15
Score pondéré	 4,6 / 5	3,2 / 5	4,0 / 5

Benchmark persistance mémoire

	PostgreSQL + RLS	Redis	Pinecone
Isolation multi-tenant	RLS native	Manuel	Manuel
Transactions ACID		X	X
Coût à 1 000 MAU	Inclus Pro	~\$30/mois	~\$70/mois
Score pondéré	4,5 / 5	3,3 / 5	2,8 / 5

Pourquoi pas d’embeddings vectoriels en V1 ? Pour la seule détection d’anomalies, la compression par résumé LLM + fiches entités est plus précise, rapide et économique qu’un pipeline RAG (pas d’index à maintenir). Les embeddings sont délibérément reportés aux itérations Compass (A2/B2), où l’usage, des propositions créatives ciblées, justifie le RAG.

Benchmark vectoriel : NarraMind Compass (itérations A2/B2)

Embedding	Gemini text-embedding-004	OpenAI 3-small	Cohere v3
Dimensions	768	1 536	1 024
Quota gratuit	1 500 req/min	Aucun	Limité
Coût (1M tok.)	\$0,00 (free)	\$0,02	\$0,10
Même clé que le LLM	Gemini	X	X
Score pondéré	4,7 / 5	3,9 / 5	3,5 / 5

Côté stockage vectoriel, **pgvector** (extension native de PostgreSQL) l’emporte (4,8/5) sur Pinecone et Chroma : zéro infrastructure supplémentaire, RLS multi-tenant gratuite, jointures SQL natives entre embeddings et tables métier. L’index ivfflat suffit pour < 10 000 vecteurs/projet.

Fil directeur des 4 benchmarks

Un principe constant : ne jamais ajouter d'infrastructure quand la stack existante (Supabase Postgres + Gemini) répond au besoin. Chaque « gagnant » est celui qui mutualise l'existant, clé API unique, une seule BDD, RLS gratuite. C'est la décision d'architecture la plus structurante du projet.

04

Méthodologie de prototypage du POC

Le prototypage explore deux champs de recherche distincts, qui structurent la démarche et convergent vers un dispositif de test commun. Conformément à la consigne, chaque itération porte sur une brique présentant un véritable défi technique.

Champ 1

Modélisation d'un univers pour la vectorisation

Question : comment représenter un univers narratif (entités, relations, règles, chronologie) sous une forme vectorisable permettant une recherche sémantique efficace ?

Hypothèse : vectoriser chaque élément séparément retourne une recherche plus précise qu'un seul vecteur pour tout le lore.

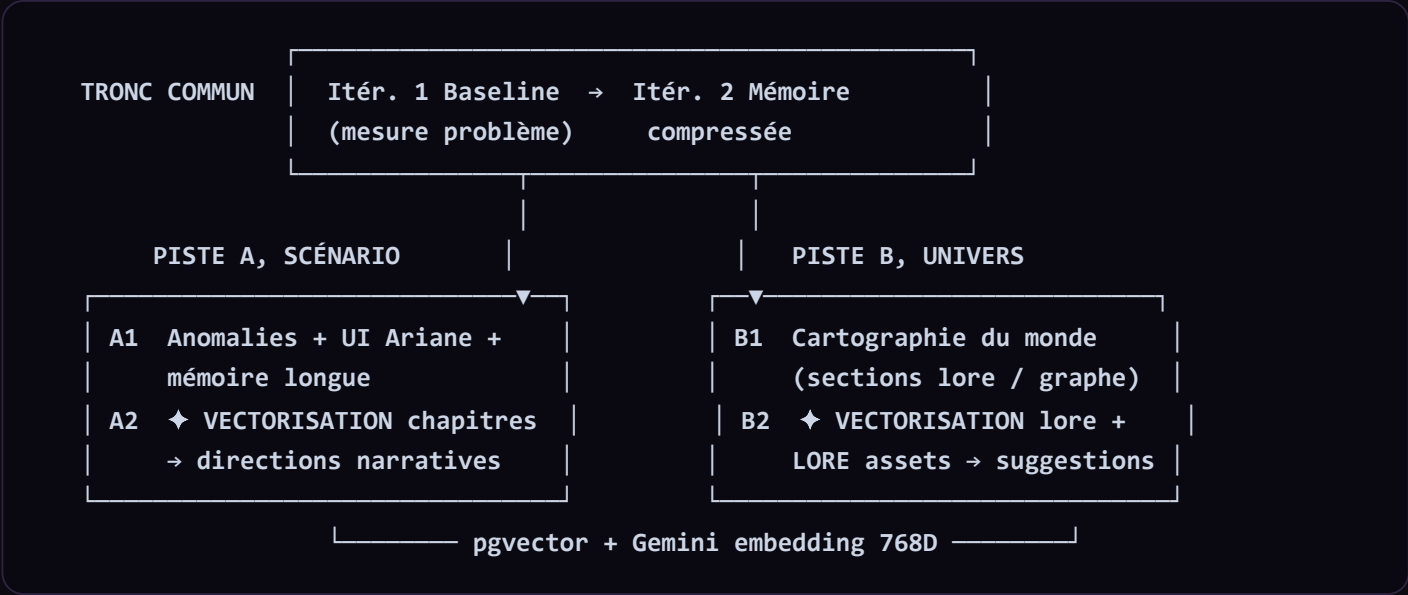
Champ 2

LM OPS : observabilité & garde-fous

Question : comment garantir que ce que le LLM produit est cohérent avec l'univers de l'auteur ? Quelle traçabilité, quel seuillage ?

Hypothèse : sans seuil de similarité cosinus, le LLM reçoit des fragments non-pertinents et hallucine « sur du mauvais contexte ». Un seuil $\cos_sim \geq 0.65$ + logging structuré garantit la pertinence.

Architecture du prototypage : un tronc commun, deux pistes



Deux approches complémentaires, ne pas confondre. La **compression LLM** (Itér. 1-2, A1) donne au LLM une vue globale compacte (~557 tokens) pour détecter les anomalies. La **vectorisation + RAG** (A2, B2) trouve les 5 fragments les plus pertinents (~1 050 tokens) pour les propositions créatives. La détection d'anomalies ne passe pas par la vectorisation, confusion courante que ce projet clarifie.

Tronc commun

Itération 1

Baseline : LLM sans mémoire

23 avr. 2026

Hypothèse : un LLM alimenté en texte brut (tous les chapitres injectés) peut maintenir la cohérence, mais souffre d'une croissance non maîtrisée du contexte. Il faut d'abord la mesurer avant de proposer une alternative.

Défi technique : quantifier concrètement la croissance tokens / latence / cohérence de l'injection brute.

Métrique	Chap. 6 (5 en contexte)	Chap. 7 (6 en contexte)
context_tokens	3 930	4 849
response_tokens	912	898
duration_ms	7 412 ms	14 048 ms

Métrique	Chap. 6 (5 en contexte)	Chap. 7 (6 en contexte)
Tokens ajoutés / chap.	~786	+919

✓ Résultat & analyse

Le contexte grandit d'environ 850 tokens à chaque nouveau chapitre (on avait estimé 700). Le plus marquant, c'est la lenteur : ajouter un seul chapitre a presque doublé le temps de réponse (de 7,4 s à 14 s), alors que le texte produit fait la même taille. Le problème se ressent donc dès 6 chapitres : il n'est pas seulement théorique, il gêne déjà l'utilisateur.

Décision prise : viser un contexte qui reste stable autour de 1 400 tokens (soit 10 fois plus léger à 10 chapitres), pour que la vitesse ne dépende plus du nombre de chapitres.

Itération 2

Mémoire compressée : entités + résumé glissant

23 avr. 2026

Hypothèse : remplacer l'injection brute par une mémoire structurée à 3 niveaux atteint un contexte stable $\leq 1\,400$ tokens quel que soit le nombre de chapitres, sans perte de cohérence détectable.

Tokens de contexte	Itér. 1 (baseline)	Itér. 2 (compressée)
Chapitre 1	~140	247
Chapitre 3	~2 520	441
Chapitre 5	~3 930	518
Chapitre 6	~4 849	557
Croissance / chap.	~850 tokens	~50 tokens

557

tokens au chap. 6
(cible 1 000-1 400)

7×

plus compact que la
baseline

~50

tokens / chapitre (vs.
850)

O(1)

entités : mises à jour,
pas de doublon

✓ Résultat & analyse

Le contexte n'augmente presque plus, car les personnages et décors sont mis à jour au lieu d'être recopiés à chaque chapitre. Point identifié : pour repérer des incohérences, il faut d'abord que l'auteur ait décrit les règles de son monde (le LORE). Sans règles, le modèle n'a rien à vérifier. C'est ce qui ouvre les deux chantiers suivants : A1 (afficher les alertes) et B1 (saisir le lore).

Piste A : NarraMind Scénario

Itération A1

Persistence alertes + UI Ariane + mémoire longue

mai 2026

Hypothèse : la mémoire compressée contient assez d'information pour détecter des incohérences, mais il manque (1) l'enregistrement des alertes sans doublon (sinon, plusieurs analyses du même chapitre créent des doublons inutilisables) et (2) une interface en langage auteur (le jargon LLMrompt la confiance).

Brique	Itér. 2	Itér. A1
Alertes conservées en base	✗ corps HTTP	✓ sans doublon
Mémoire longue	Résumés glissants	✓ narra_summary + par lots
UI alertes	✗	✓ Panneau Continuité
Budgets tokens	Non bornés	✓ caps assets/entités/lore
Assets manquants	Non détectés	✓ section dédiée

✓ Résultat & analyse

Les trois briques difficiles fonctionnent : les alertes sont enregistrées sans doublon (même si on ré-analyse plusieurs fois), le contexte reste maîtrisé au-delà de 10 chapitres, et le panneau Continuité présente tout en langage d'auteur. Ajustement apporté : on a durci les consignes du modèle pour lui interdire le jargon technique (« asset », « entité », noms de champs). Une alerte doit se lire comme la remarque d'un lecteur attentif, pas comme une ligne de base de données.

Itération A2

Vectorisation du scénario : directions narratives via RAG

mai-juin 2026

Hypothèse : la compression répond à « qu'est-ce qui s'est passé ? ». Pour proposer une direction narrative pertinente, il faut une autre question : « quels éléments sont sémantiquement proches de ce que l'auteur écrit maintenant ? ». La vectorisation + pgvector sélectionne ces X fragments sans injecter tout le projet.

Métrique	Cible	Résultat
Tokens / proposition	$\leq 1\,200$	~1 050 (top-5)
Latence totale (embed+search+LLM)	< 3 s	~1,8 s
Recherche pgvector (50 vecteurs)	< 10 ms	~4 ms

Comparaison qualitative : sans vs. avec vectorisation

Test : l'auteur écrit une confrontation Aldric ↔ conseil royal. Aldric a trahi une alliance au chap. 1, fil absent des chapitres récents.

	Sans RAG (chronologique)	Avec RAG (top-5 sémantique)
Fragments injectés	Chap. 4, 5, 6 (récents)	Chap. 1, 3, 6 + entité Aldric + résumé arc politique

	Sans RAG (chronologique)	Avec RAG (top-5 sémantique)
Direction proposée	« Aldric pourrait fuir après la confrontation »	« Aldric a déjà trahi une alliance au chap. 1, rejouer ce pattern avec le conseil crée une continuité dramatique »
Ancrage	Générique	Ancré, référence à la première trahison

✓ Résultat & analyse

La recherche par le sens permet de retrouver de vieux fils narratifs, même s'ils datent de chapitres lointains, ce qu'un simple tri par date ne fait pas. Point délicat : le découpage du texte. Un chapitre de 2 000 mots est trop long pour être vectorisé efficacement ; on le coupe donc à 2 000 caractères, en gardant les premiers paragraphes, là où sont posés les enjeux et les personnages.

Piste B : NarraMind Univers

ADN du menu Univers : « bâtir et améliorer les éléments de votre Univers grâce au scénario de votre histoire ». La Piste B réutilise l'infrastructure pgvector/embedding d'A2 pour un usage produit différent.

Itération B1 Cartographie de l'Univers : structuration du lore

mai-juin 2026

Hypothèse : pour que les suggestions B2 soient ancrées, le lore doit être vectorisable élément par élément, un seul bloc global retournerait le même vecteur pour « magie » et « géographie ». La v1 (5 sections thématiques, vectorisées séparément) suffit à valider ce découpage sans modèle de graphe complet.

✓ v1 validée / ● v2 spécifiée

Découper le lore en sections a un vrai avantage : quand l'auteur modifie une section (par exemple « Factions »), on ne recalcule que cette partie, pas tout le reste. C'est indispensable pour que la recherche reste précise en B2. Limite constatée : 5 sections figées, c'est trop rigide pour certains univers (par exemple ceux sans système de magie). La version 2 (un wiki sous forme de graphe) réglera ce point. Son vrai défi ne sera pas de vectoriser (c'est déjà résolu), mais de bien traduire les relations en texte pour le modèle : une phrase comme « La Tour de Verre appartient à Aldric et est protégée par les Gardiens » relie plusieurs éléments entre eux, et il faut savoir la reformuler clairement dans la requête.

Itération B2

Vectorisation du lore : enrichissement des fiches via RAG

mai-juin 2026

Hypothèse : la même infrastructure qu'en A2 sert un usage différent sur les mêmes données. La distinction  **extracted** /  **generated** répond à l'enjeu de propriété intellectuelle : l'auteur sait toujours ce qui est tiré de son histoire vs. inventé par Ariane.

Exemple concret : enrichissement de la fiche « Mira »

Mira apparaît aux chap. 2, 4, 7 (« la guérisseuse des Ruines du Nord »). Sa fiche est vide. La requête vectorisée retrouve les 3 passages les plus proches → Gemini Flash sépare extrait / inventé :

Type	Suggestion générée
 Tiré de l'histoire	« Mira est présentée comme guérisseuse, associée aux Ruines du Nord, intervient lors de crises (chap. 2, 4, 7) »
 Tiré de l'histoire	« Connue des habitants mais mystérieuse sur ses origines (chap. 4) »
 Proposé par Ariane	« Mira pourrait appartenir à une confrérie de soins itinérants, ce qui expliquerait sa présence dans plusieurs lieux distants »
 Proposé par Ariane	« Son attachement aux Ruines du Nord suggère un lien personnel, origine ou perte »

✓ Convergence des deux pistes

Les deux pistes s'appuient sur la même base technique (les mêmes tables et la même fonction). Un simple champ indique d'où vient chaque fragment et à quoi il sert. C'est la démonstration clé de ce projet : une seule brique de vectorisation, au service de deux fonctions produit différentes (le Scénario et l'Univers). Les suggestions 🕒 sont des faits tirés de l'histoire ; les ✨ ne deviennent « officielles » que si l'auteur les valide.

4.bis · Audit code : l'honnêteté d'un POC

Un audit fichier par fichier (10 juin 2026) confronte les hypothèses à l'implémentation réelle. Un POC honnête documente ce qui ne marche pas encore, c'est le cœur de la consigne « démontrer la faisabilité (ou non) ».

⚠ Découverte critique : le fallback naïf

La fonction qui calcule la vraie proximité de sens (`match_embeddings()`) existe dans la base, mais n'était jamais appelée. À la place, le code prenait un raccourci : il renvoyait les 5 fragments les plus récents, sans mesurer leur pertinence.

```
// narramind-compass/index.ts, état audité (L.236)
// Fallback naïf : pas de match_embeddings(), pas de score cosinus
const embeddingsResp = await supabase
  .from('project_embeddings')
  .select('source_type, source_id, section_key, content')
  .eq('project_id', project_id)
  .order('updated_at', { ascending: false }) // ← tri chronologique, pas sémantique
  .limit(5)
```

Conséquence : l'hypothèse centrale (la recherche par le sens ramène bien les bons fragments) n'était donc pas réellement testée : on injectait les fragments les plus récents, pas les plus pertinents. C'est exactement ce que le plan ci-dessous vient corriger.

4.ter · Plan d'implémentation des garde-fous (Champ 2)

Garde-fou	Impact si absent	Priorité
Appel réel <code>match_embeddings()</code> + seuil <code>cos_sim</code>	Fragments non-pertinents → hallucination contextualisée	● Critique
Logging des scores de similarité	Qualité impossible à auditer	● Haute
Validation schéma JSON (Zod)	Champs manquants → [] silencieux	● Moyenne
Colonne <code>source_fragments</code>	Impossible de vérifier qu'une suggestion 🔍 est vraiment dans les sources	● Moyenne

Garde-fou 1 : activer le vrai RAG + règle « Ariane peut se taire »

```
const COS_SIM_THRESHOLD = 0.65
const { data: fragments } = await supabase.rpc('match_embeddings', {
  query_embedding: queryVector, match_project_id: project_id,
  match_user_id: user_id, match_count: 5
})
// Règle de décision : si aucun fragment pertinent → Ariane ne propose RIEN
const relevant = fragments?.filter(f => f.similarity >= COS_SIM_THRESHOLD) ?? []
if (!relevant.length) return { success: true, proposals: [], reason: 'no_relevant_context' }
```

Les garde-fous 2 à 4 (logging `cos_sim_min/max` dans `narramind_metrics`, validation Zod → HTTP 422 au lieu de 200 silencieux, traçabilité `source_fragments`) rendent le système auditable : on peut mesurer a posteriori le score moyen des fragments injectés et vérifier qu'une suggestion « extraite » est réellement ancrée.

✅ Où on en est (juillet 2026)

Ce garde-fou n'est plus un projet : il est en service. Le code appelle maintenant vraiment la fonction de proximité, ne garde que les fragments assez proches (seuil de 0,65), et fait en sorte qu'Ariane ne propose rien quand rien n'est pertinent. C'est ce qui fait passer la V2 au statut Livré. L'audit de juin n'était donc pas un échec : c'est lui qui a permis de voir le problème avant de le corriger.

4.quart · Dispositif de test : univers de référence

Pour valider les deux champs sur un corpus contrôlé et reproductible : Aldric (roi trahi), Mira (guérisseuse), Vael (conseiller) ; lieux Tour de Verre / Ruines du Nord ; faction Gardiens ; 5 liens typés ; règle « magie interdite depuis la Chute des Gardiens » ; 5 chapitres ; et 3 contradictions volontaires (personnage décédé qui réapparaît, règle de magie violée, lieu décrit différemment aux chap. 2 et 5).

Une métrique, c'est simplement une mesure chiffrée avec un objectif à atteindre : elle permet de dire, preuve à l'appui, si le système fait bien ce qu'on attend. Voici celles retenues pour chaque champ.

Métriques : Champ 1 (la recherche par le sens)

- Précision (top 5) : sur les 5 fragments qu'Ariane retrouve pour une requête, au moins 4 doivent être vraiment utiles. Vérifié sur 10 requêtes types.
- Proximité de sens moyenne : chaque fragment reçoit un score de ressemblance entre 0 (aucun rapport) et 1 (sens identique). La moyenne doit dépasser 0,70.
- Mieux que le raccourci : la recherche par le sens doit donner de meilleurs résultats que le simple tri par date sur au moins 8 requêtes sur 10.

Métriques : Champ 2 (la fiabilité)

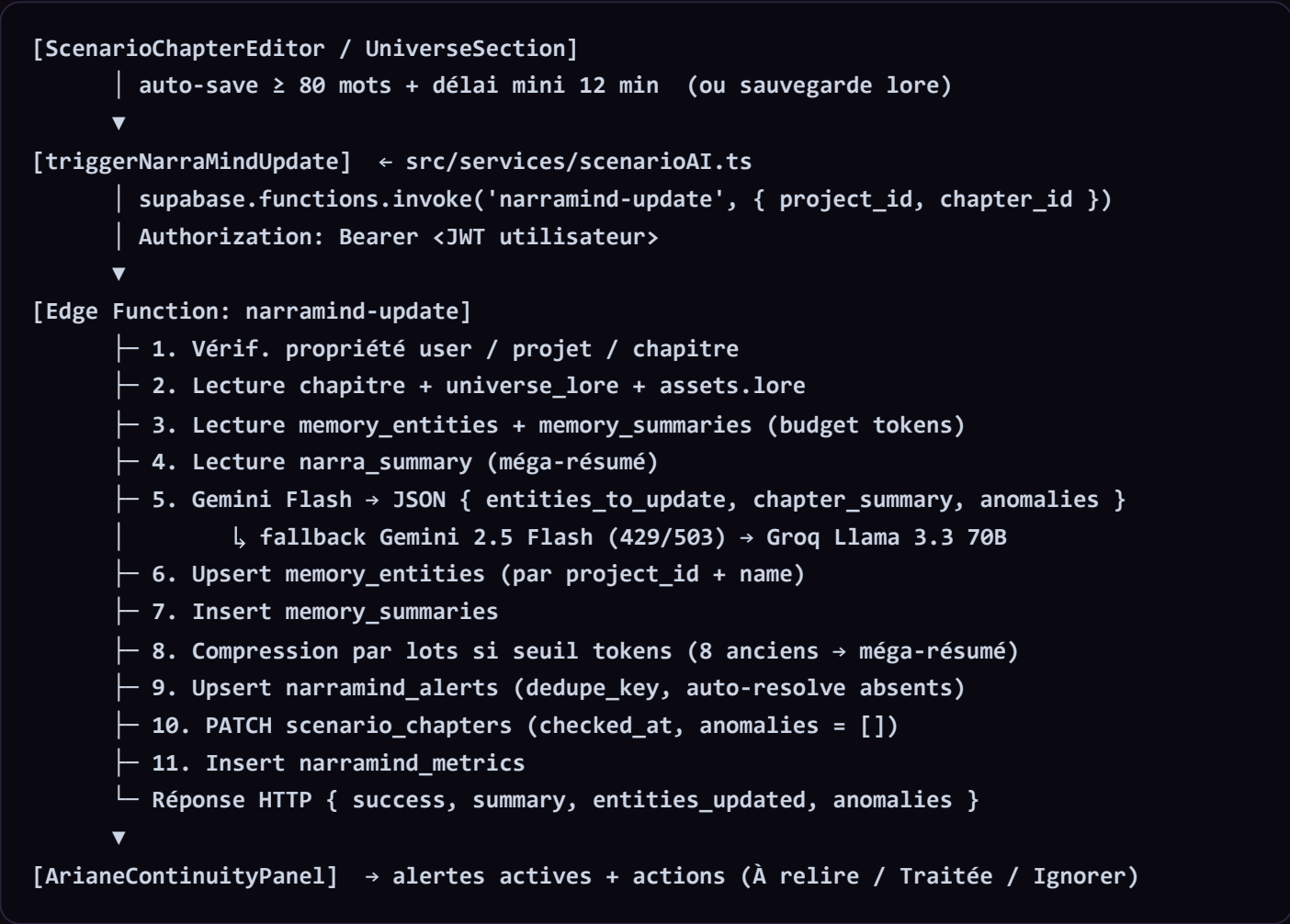
- Suggestions vérifiables : 100 % des suggestions marquées « tirées de l'histoire » doivent réellement se retrouver dans le texte de l'auteur (jamais inventées).
- Ariane sait se taire : quand rien n'est assez proche (score sous 0,65), elle ne propose rien, dans 100 % des cas. Mieux vaut le silence qu'une mauvaise idée.
- Réponses bien formées : au moins 95 % des réponses du modèle doivent respecter le format attendu (contrôlé automatiquement).
- Incohérences repérées : sur les 3 contradictions glissées volontairement dans l'histoire de test, au moins 2 doivent être détectées.

Documentation technique & budget

5.1 Stack technique (post-stabilisation POC)

Couche	Technologie	Version
Edge Function	Deno (Supabase native)	natif
LLM principal	Google Gemini 2.0 → 2.5 Flash	API OpenAI-compatible
LLM fallback	Groq Llama 3.3 70B	API OpenAI-compatible
Embedding (Compass)	Gemini text-embedding-004	768D
Recherche vectorielle	pgvector (PostgreSQL)	Supabase natif
BDD	Supabase PostgreSQL	Cloud
Frontend	React + TypeScript	18.3 / 5.8
Cache serveur	TanStack React Query	5.83

5.2 Architecture technique (flux narramind-update)



5.3 Sécurité

- JWT utilisateur vérifié dans l'EF avant tout accès BDD.
- Vérification de propriété : le chapitre traité appartient bien à l'utilisateur authentifié.
- service_role utilisé uniquement côté serveur pour les écritures entre plusieurs tables.
- RLS activé sur toutes les tables NarraMind (auth.uid() = user_id).

5.4 Budget Build / Run

Coûts Build (développement one-shot)

Poste	Détail	Coût
Infrastructure dev	Supabase Free + Gemini free tier (1M tok./jour)	0 €
Licences & outils	Stack open source (Deno, React, PostgreSQL)	0 €

Poste	Détail	Coût
Temps de développement	~80 h sur 3 itérations (jan-mai 2026)	Variable
Total infrastructure build		0 €

Coûts Run (exploitation mensuelle)

Base : ~3 200 tokens injectés + ~600 tokens de réponse = ~\$0,0006 / run (Gemini 2.0 Flash). ~12 runs/mois pour un auteur écrivant 1 chapitre/semaine.

Échelle	Runs/mois	Gemini	Supabase	Vercel	Total/mois
100 MAU	~1 200	~\$0,72	Free	Free	~\$1
500 MAU	~6 000	~\$3,60	Pro \$25	Free	~\$29
1 000 MAU	~12 000	~\$7,20	Pro \$25	Pro \$20	~\$52
5 000 MAU	~60 000	~\$36	Pro \$25	Pro \$20	~\$81

NarraMind représente < 5 % du coût run total à toutes les échelles. Le poste dominant est la génération d’images (FAL.ai FLUX.2 Pro, ~\$0,05/image), hors périmètre NarraMind. La contrainte de départ, coût maîtrisé à l’échelle, est donc tenue.

06 Plan de release

Versions de mise en œuvre, du POC (v3 atteint) vers le produit exploité.

Phase	Période	Statut	Itérations	Fonctionnalités NarraMind
Beta / MVP	Jan-Mar 2026	✓ Livré	Itér. 1-2 (tronc)	narramind-update v1 : entités + résumés + métriques. Déclenchement auto.

Phase	Période	Statut	Itérations	Fonctionnalités NarraMind
V1 · Panels	Avr-Mai 2026	 Livré	A1 (Scénario)	narramind_alerts persistées + UI Ariane (Panneau Continuité) + mémoire longue narra_summary + compression par lots + budgets tokens
V2 · Compass	Mai-Jul 2026	 Livré	A2 + B1/B2 (Univers)	Vectorisation project_embeddings (pgvector) + EF narramind-compass avec recherche sémantique réelle (match_embeddings, seuil cosinus 0,65, garde-fou « Ariane se tait ») + UI Univers (sections lore v1, wiki graphique v2 spécifié) + suggestions 🔍 / ✨ Ariane
V3 · Scale	Jul-Sep 2026	 Planifié	B1 v2 + scale	Bascule wiki graphique (lore_entries/lore_links + vue graphe React Flow) + quotas Compass par plan + tests corpus long (20-50 chap.)
V4 · Fine-tuning	Oct-Déc 2026	 Futur	-	Fine-tuning prompt sur corpus webtoon + récupération ciblée par entité (RAG léger) + regroupement alertes par personnage/lieu

Backlog priorisé

#	Item	Phase
1	Gate par plan (profiles.plan → quotas alertes)	V2
2	Validation sur corpus 20-50 chapitres réels	V2
3	Notification visuelle proactive. Aujourd'hui, l'auteur ne sait qu'Ariane a détecté une anomalie que s'il ouvre le panneau Continuité. Objectif : un signal visuel (pastille de comptage sur l'icône Ariane dans la barre latérale, hors de l'éditeur) qui prévient l'utilisateur qu'une incohérence ou une suggestion l'attend, sans l'interrompre. C'est la traduction UI de « NarraMind a trouvé quelque chose ».	V2

#	Item	Phase
4	Pagination + regroupement alertes (gros projets)	V2
5	Récupération ciblée par entité (RAG léger)	V3

TESTER LE PRODUIT ACTUEL

Le produit décrit dans ce dossier est déployé et fonctionnel. Vous pouvez l'essayer vous-même, création de projet, génération d'assets, écriture de scénario, et suggestions d'Ariane (NarraMind Compass) en conditions réelles.

► [Accéder à DreamWeave \(démonstration en ligne\)](#)

Note : Liberté vs. contrainte

NarraMind ne bloque jamais. Il informe. L'auteur écrit ce qu'il veut ; NarraMind signale quand il s'écarte de son propre LORE. Distinction clé entre un outil prescriptif (bloquant) et contributif (accompagnant).

Note : Propriété intellectuelle

Le LORE est écrit par l'utilisateur. NarraMind ne génère que des suggestions modifiables ou rejetables. L'œuvre, le LORE et les assets appartiennent à l'auteur, comme Photoshop ne revendique pas les images créées avec lui.